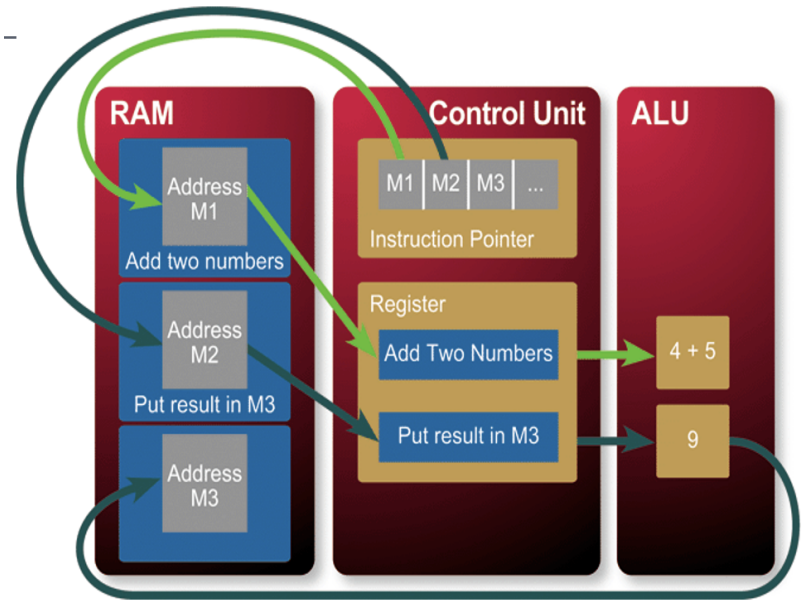
Process

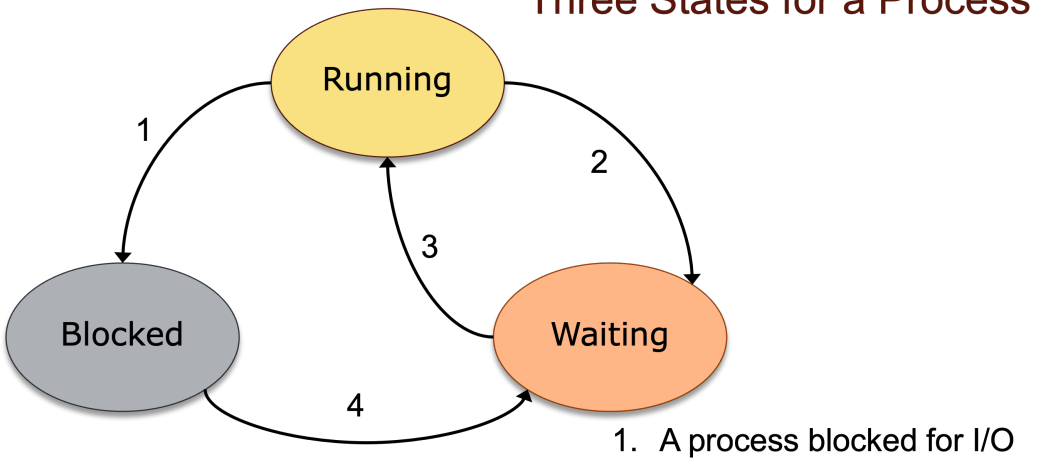
What is process?

- A process is a program in execution.
- It is associated with:
  - Address space – memory location of program instructions and data.
  - Set of registers (a smaller piece of memory built in CPU).
  - All other information for executing the process.
- Based on the Von Neumann Computer Architecture (I/O part is omitted here):

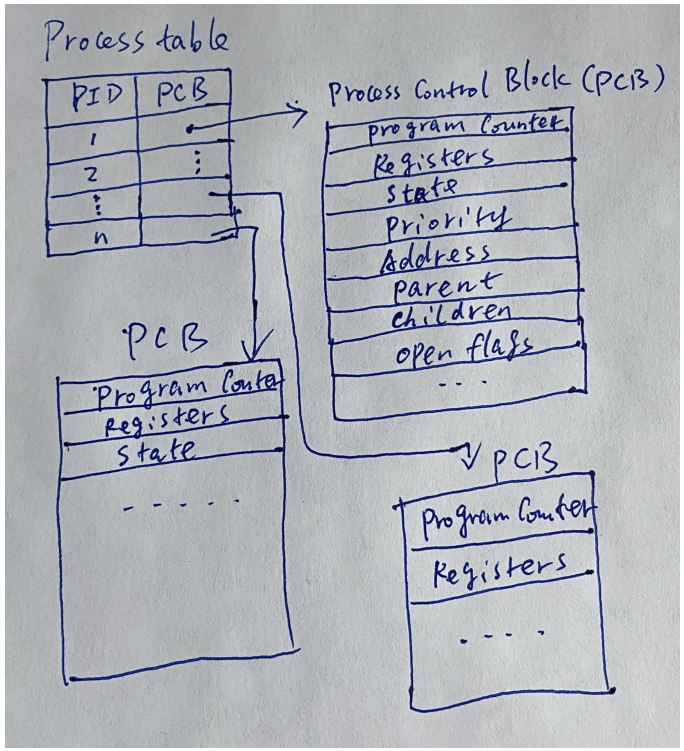


The control unit's instruction pointer indicates M1, a location in memory. The control unit fetches the "Add two numbers" instruction from M1. This instruction is then sent to the ALU. The instruction pointer then changes to M2. The processor fetches the instruction located in M2, moves it to a register, and executes it

Process State and Process Table



1. A process blocked for I/O
2. A process time out
3. A process scheduled
4. A process finish it's I/O



- A **process table** keeps track of all information on each process. PCB is used to track the process's execution status, including process state, pc, open file lists, etc..
- If a process is suspended (waiting or block state), information for the snapshot of the process is saved into the process table.
- Once a process resumes a CPU time, all information for the process execution is copied back from its process table.

Process Identifiers

- When a process is created, kernel provides unique process ID (PID), a non-negative integer.
- When a process terminate, its ID becomes reusable for a newly created process.
- A few PID numbers reserved by system itself:
  - pid 0: a process scheduler
  - pid 1: the init process
- Command to show all the processes
  - Linux: `ps -e`
  - Mac: `ps ax`

Code Example for Getting PID

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(){
      printf ("Process ID = %d \n", getpid());
      printf ("Parent's ProcessID = %d \n", getppid());
      printf ("Real User's ID = %d \n", getuid());
      printf ("Effective User's ID = %d \n", geteuid());
      printf ("Real Group's ID = %d \n", getgid());
      printf ("Effective Group ID = %d \n", getegid());
      return 0;
}
```

- The `getpid()` returns the process ID of the calling process.
- The `getpgrp()` returns the process group ID of the calling process.
- The `getppid()` returns the parent process ID of the calling process.

Creating Child Process with fork() System Call

- When a process calls the `fork()`, it creates a copy of itself, and this new process is the **child process**.
- On success, `fork()`
  - returns pid `0` to the child process, which continues and uses that information to determine that it's the child;
  - it also returns a positive value, the child pid, to the parent process, which continues and uses that information to determine that it's the parent.
- If `fork()` returns a negative value, then creating a child process is a failure.
- The child process get a copy of all the data, but run in separate memory space.
- The child and parent processes only share the text segment, read-only data, and shared libraries or resources.

Example Usage of fork()

- Your shell uses fork to run the programs you invoke from the command line.
  E.g. `vim foo &`, then `ps`, you'll see the vim process in addition to the bash shell process.
- Web servers like apache use fork to create multiple server processes, each of which handles requests in its own memory space. If one dies or leaks memory, others are unaffected, so it functions as a mechanism for fault tolerance.
- Google Chrome uses fork to handle each page within a separate process. This will prevent client-side code on one page from bringing your whole browser down.

fork() System Call - Example Code 1

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void ChildProcess(); /* child process prototype */
void ParentProcess(); /* parent process prototype */

int main(void) {
  pid_t pid;
  pid = getppid(); /* get parent process ID */
  printf("pid = %d\n", pid);
  pid = fork(); /* create a child */
  if (pid == 0) /* means a child process*/
    ChildProcess();
  else
    ParentProcess();
  return 0;
}
void ChildProcess(){
  printf("*** Child is done ***\n");
}
void ParentProcess(){
  printf("*** Parent is done ***\n");
}
```

Output after compiling and running the executable:

```
pid = 10014
*** Parent is done ***
// pid 0
*** Child is done ***
```

fork() System Call - Example Code 2

```
#include<stdio.h>
#include<string.h>
#include<unistd.h>
#define   MAX_COUNT  2

/* Notice how the parent and child got different pid
after fork, and each run the for loop MAX_COUNT times.
*/
int  main(void) {
     pid_t  pid, ppid;
     int    i;
     ppid = getpid(); /* this is parent process ID */
     printf("ppid=%d\n", ppid);
     fork(); /*create a child */
     for (i = 1; i <= MAX_COUNT; i++) {
           pid = getpid();
           printf("pid=%d, i=%d\n", pid, i);
     }
     return 0;
}
```

Output after compiling and running the executable:

```
ppid=10222
pid=10222, i=1
pid=10222, i=2
pid=10223, i=1
pid=10223, i=2
```

fork() System Call - Example Code 3

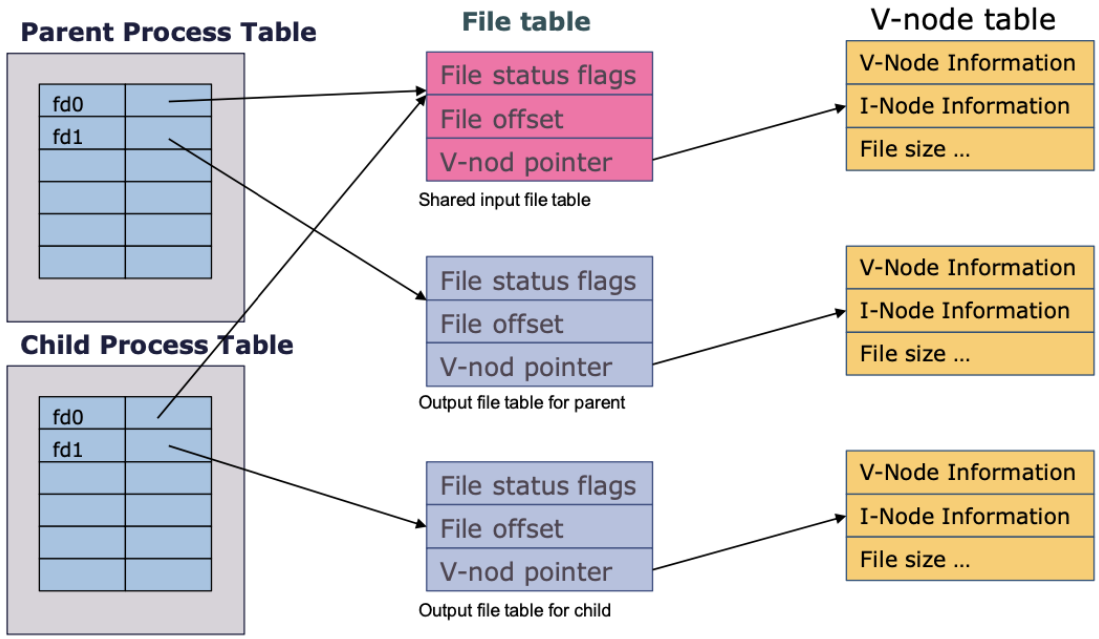
```
#include<stdio.h>
#include<string.h>
#include<unistd.h>
#define   MAX_COUNT  2
#define   BUF_SIZE   50
int  main(void) {
     pid_t  pid, ppid;
     int    i;
     char   buf[BUF_SIZE];
     ppid = getpid(); /* this is parent process ID */
     fork(); /*create a child */
     for (i = 1; i <= MAX_COUNT; i++) {
           pid = getpid();
           if (pid == ppid)/* parent works here */ {
              sprintf(buf, "Parent(%d) process executed %d times\n", ppid, i);
              write(1, buf, strlen(buf));
           }
           else /* child work here */{
              sprintf(buf, "Child(%d) process executed %d times\n", pid, i);
              write(1, buf, strlen(buf));
           }
     }
     return 0;
}
```

Output after compiling and running the default executable `./a.out`

```
Parent(10298) process executed 1 times
Parent(10298) process executed 2 times
Child(10299) process executed 1 times
Child(10299) process executed 2 times
```

fork() Application in File Sharing

Consider a process that opens an input file, then forks a child process, then both parent and child process starts to write to their own output file separately. Given that they will share the input file table, including offset, what would be the results in their output files without synchronization mechanism between them?



Race Condition

- Occurs when multiple processes or threads read and write data items.
- The final result depends on the order of execution – The “loser” of the race is the process that updates last and will determine the final value of the variable.
- E.g. `b=1, c=2` are global variables shared by P1 and P2; then, we have:
  scenario 1: `b=3, c=5`
  scenario 2: `b=4, c=3`

P1	P2
b=b+c	
	c=b+c

P1	P2
	c=b+c
b=b+c	

Process Termination

There are five ways to terminate a process:

- Normal Termination
  - Return from main function
  - Call `exit()`: perform certain clean up then return control to the kernel.
  - Call `_exit()` (by child process): control immediately return to the kernel (used between child and parent processes).
- Abnormal Termination
  - Call `abort`
  - Terminate by a signal (SIGTERM or SIGQUIT)

```
#include <stdlib.h>
void exit (int status);

#include <unistd.h>
void _exit(int status);

#include <stdlib.h>
void abort(void);
```

Process Termination

- The `exit` status of a child will be used for parent process termination.
- When a child exits, the parent process will receive a `SIGCHLD` signal to indicate that one of its children has finished execution; the parent process will typically call the `wait()` system call at this point.
- That call will provide the parent with the child's exit status, and will cause the child to be removed from the process table
- When a process terminate either normally or abnormally, the kernel sent a signal (SIGCHLD) to a parent.
- The parent can ignore the signal or call `wait` or `waitpid` to take care the signal.

```
#include <sys/wait.h>
pid_t wait (int *status);
pid_t waitpid(pid_t pid, int *status, int option);
```

wait() System Calls

The `wait()` system call blocks a parent process from its execution till the child process terminates.

```
#include <sys/wait.h>
pid_t wait (int *status);
```

- `status` is an integer pointer used to access the child's exit value. If we want to ignore the exit value, we can use NULL here.
- Upon termination of the child process, the parent process returns the pid of the child process and its exit status using the macro function `WEXITSTATUS()`; -1 on error.
- If there is no child process running, then `wait()` has no effect.

wait() System Call Code Example

```c
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <unistd.h>
int main(){
    // create a child process
    int child = fork();
    int exitStatus;
    int childPid;
    // code for the child process
    if(child == 0){
        // display running message and make child sleep
        printf("Child: I am running!!\n\n");
        printf("Child: I have PID: %d\n\n", getpid());
        sleep(4);
        // child exits with code 50
        exit(50);
    }   // code for the parent process
    else{
        // print parent message
        printf("Parent: I am running and waiting for child to finish!!\n\n");
        // call wait system call
        childPid = wait(&exitStatus);
        // print the msg of the child process
  printf("Parent: Child of PID: %d done, exit Status: %d\n\n", childPid, WEXITSTATUS(exitStatus));
    }
    return 0;
}
```

waitpid() System Calls

The `waitpid` system call allows parent process to wait for a specific child to terminate before parent process terminates.

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int option);
```

- `pid` refers to the pid that the parent wants to wait for. If the provided pid is 0, it will wait for any arbitrary child to finish.
- `status` is an integer pointer used to access the child's exit value. If we want to ignore the exit value, we can use NULL here.
- `option` refers to additional flags to modify the function's behavior. For example,
  - `WCONTINUED` is used to report the status of any child process that has been terminated and those that have resumed their execution after being stopped.
  - `WNOHANG` is used when we want to retrieve the status information immediately when the system call is executed. If the status information is not available, it returns an error.
  - `WUNTRACED` is used to report any child process that has terminated.
- If success, the function returns pid of the terminated child process; -1 on error.

waitpid() System Calls Code Example

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
int main(){
    int cpid = fork();
    int cpid2 = fork();
    if(cpid != 0 && cpid2 != 0){
        int waitPID = 0;
        int status;
        printf("\nParent: I am going to wait for the process with process ID: %d\n", cpid2);
        while(waitPID == 0){
            waitPID = waitpid(cpid2, &status, WNOHANG);
        }
        printf("\nParent: Waited for child, the return value of waitpid(): %d\n", waitPID);
        printf("\nParent: The exit code of terminated child: %d\n", WEXITSTATUS(status));
        exit(1);
    }
    else if(cpid == 0 && cpid2 != 0){
        printf("\nChild1: My process ID is: %d, and my exit code is 1\n", getpid());
        exit(1);
    }
    else if(cpid != 0 && cpid2 == 0){
        printf("\nChild2: My process ID is: %d, and my exit code is 2\n", getpid());
        exit(2);
    }
    else{
        printf("\nChild3: My process ID is: %d, and my exit code is 3\n", getpid());
        exit(3);
    }
    return 0;
}
```

system() System Calls

- We can execute bash commands inside c program by using system() system call.
- The `system()` is implemented by calling `fork(), exec`, and `waitpid`, there are three types of return value
  - `-1`: If either the fork() or waitpid() failed.
  - `127`: if the exec failed
  - `WUNTRACED` is used to report any child process that has terminated.
  - Other: the termination status of `waitpid()`

system() System Calls Code Example

```c
/* systemtest.c: demonstrate system() system call */
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>

int main(){
   int status;

   if ( (status = system("date")) < 0) {
      printf("system() ERROR\n");
      exit(1);
   }
   printf ("Status = %d\n", status);
   /*non existed command */
   if ( (status = system("nosuchcommand")) < 0) {
      printf("system() ERROR\n");
      exit(2);
   }
   printf ("Status = %d\n", status);
   if ( (status = system("who; exit 44")) < 0) {
      printf("system() ERROR\n");
      exit(3);
   }
   printf ("Status = %d\n", status);
   exit(0);
}
```